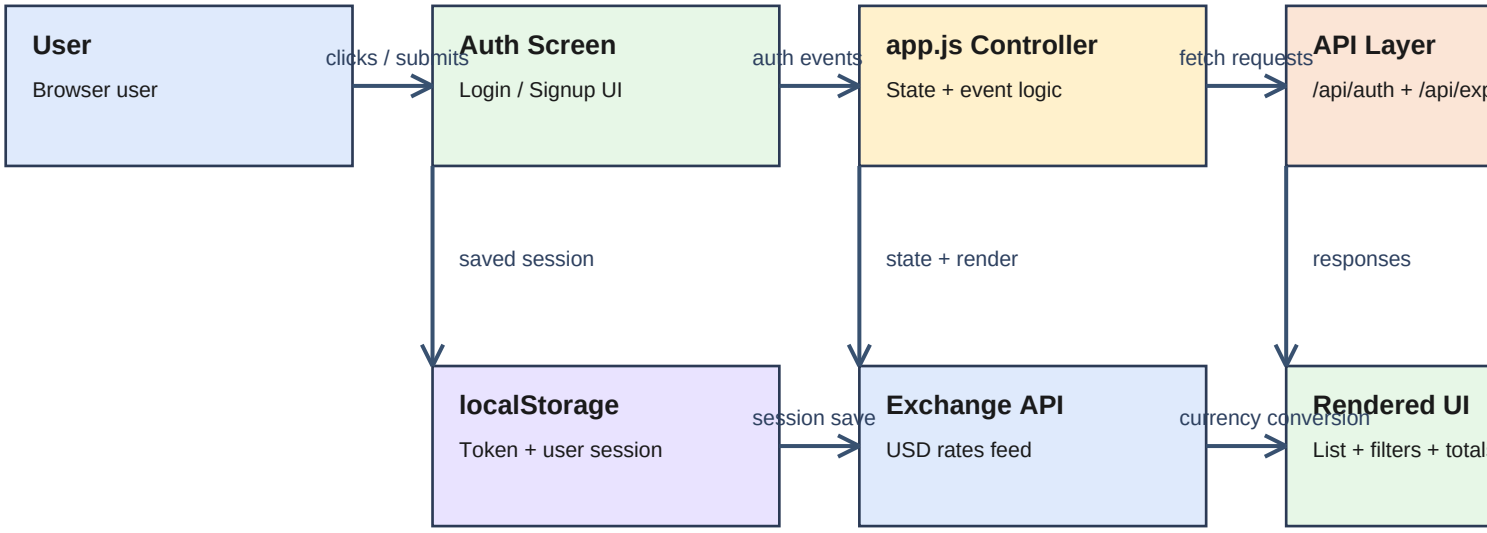
Expense Tracker System Architecture



How the system works

1. The app loads and checks localStorage for a saved token and user profile.
2. If a valid session exists, restoreSession() opens the Expense dashboard; otherwise the auth screen is shown.
3. handleAuthSubmit() validates the email and password before posting to /api/auth/login or /api/auth/signup.
4. setSession() saves the token and user so the browser remembers the user after refresh.
5. loadExpensesForCurrentUser() fetches the user's expenses and updates the list in the UI.
6. renderExpenses() filters by category, converts amounts, and recomputes totals in the home currency.
7. loadExchangeRates() fetches live FX values; if the API fails, fallbackRates keeps the app working.
8. handleDeleteExpense() deletes an item and refreshes the visible list without reloading the page.

Why these parts were chosen

- localStorage keeps the user session active without a full backend session management layer for this project.
- fetchJson() centralizes all API requests and adds the Authorization header when needed.
- convertCurrency() and loadExchangeRates() support multi-currency expense tracking and real-world use.
- renderExpenses() separates data from presentation, making the interface easier to maintain.
- Event listeners connect the UI to login, logout, add-expense, filter, and delete actions.
- The expense template makes repeated list entries consistent and easy to style.

Designed for presentation: architecture + flow + code purpose summary